

Dynamic Load Balancing for a Real-Time Analytics Dashboard:

A Comparative Evaluation of Round-Robin, Least-Connections, and Latency-Aware Routing

Ana Maria Floristean, Razvan-Constantin Croitoru
Faculty of Computer Science
Alexandru Ioan Cuza University, Iași, Romania
anaflois23@gmail.com, croitorurazvan1@gmail.com

Abstract—Distributed systems that serve real-time analytics must spread incoming requests across multiple backend workers without creating hotspots or causing cascading failures. In this paper we design, implement, and evaluate a fully containerised load-balancing testbed featuring three worker services — two fast workers and one intentionally degraded worker with a $4\times$ slowdown factor — and a custom reverse proxy supporting three routing policies: static round-robin, least-connections, and a latency-aware strategy based on Exponentially Weighted Moving Average (EWMA) scoring. We evaluate the system under four controlled workloads: constant load, traffic spike, progressive stress to 300 virtual users, and deliberate worker failure injection.

Results demonstrate clear differentiation between algorithms. Under constant moderate load (30 VUs), the latency-aware algorithm routes only 4.9% of traffic to the slow worker (versus 33% for round-robin), reducing average latency by 30% and increasing throughput by 19%. Under extreme stress (300 VUs), both adaptive algorithms sustain over 229 req/s with under 2% errors, while round-robin achieves only 173 req/s with 7.6% errors and drives all three workers into an unhealthy state. In the failure-injection scenario, the system detects the crashed worker within one health-check interval and reroutes traffic with just 0.01% total failures (1 of 9252 requests). We additionally document two implementation correctness issues — EWMA contamination by health-check latency and a concurrency race on the in-flight counter — whose presence caused all algorithms to behave identically, and whose resolution was necessary to observe the expected differentiation.

Index Terms—load balancing, round-robin, least-connections, EWMA, latency-aware routing, distributed systems, containerisation, FastAPI, k6, Docker Compose, performance evaluation, fault tolerance

I. INTRODUCTION

Modern web applications are typically deployed as a collection of stateless backend services replicated across multiple workers to achieve horizontal scalability. A *load balancer* sits in front of these workers and decides, for each incoming request, which worker should handle it. The decision policy — the *routing algorithm* — directly affects user-perceived latency, system throughput, and resilience to failures.

The simplest policy, round-robin (RR), distributes requests evenly in order. This is optimal when all workers are identical and have the same capacity. In practice, heterogeneity is common: workers run on different hardware generations, experience garbage-collection pauses, share CPU with co-located services, or become partially degraded due to software bugs or resource exhaustion. Under these conditions, a static algorithm wastes capacity by over-routing to slow workers.

Dynamic policies address this by incorporating real-time signals. Least-connections (LC) uses the current request queue depth; latency-based policies use measured response times. Both signal sources are imperfect: connection counts do not distinguish between a fast worker with many small requests and a slow worker with few large ones; response-time estimates must be tracked carefully to avoid convergence to stale values.

Research question. *Which routing strategy provides the best trade-off between throughput, tail latency, and resilience in a heterogeneous worker pool under variable workload and failure?*

To answer this, we built a reproducible containerised testbed and conducted four controlled experiments. Our contributions are:

- 1) A working implementation of three routing algorithms in a single async Python reverse proxy, including their correctness pitfalls.
- 2) A quantitative comparison across four workload regimes on real hardware, with all metrics captured from a live Prometheus endpoint and k6 summary exports.
- 3) An integrated specification connecting each system requirement to a concrete experimental measurement or functional test.
- 4) Documentation of two implementation bugs that silenced algorithm differentiation and the fixes required to restore correct behaviour.

II. BACKGROUND AND RELATED WORK

A. Load Balancing Algorithms

Load balancing has been studied since the early days of distributed computing [4]. The three canonical algorithms that form our baseline are:

Round-robin. The simplest deterministic policy; each new request is routed to the next worker in a cyclic list. It requires no state beyond a counter and performs optimally when service times are uniform [5].

Least-connections. Introduced to handle uneven service times, it routes each request to the worker with the smallest number of active (in-flight) connections. Theoretical analysis shows this is near-optimal when service times follow an exponential distribution [8]. Practical implementations track an atomic in-flight counter per worker.

Latency-based / weighted. A wider family of algorithms that use historical response-time measurements to infer worker capacity and prefer faster workers. Pai et al. [6] demonstrate that locality-aware distribution improves both throughput and cache hit rates in cluster-based servers. Envoy Proxy [7] implements an EWMA variant in production.

B. EWMA for Latency Estimation

Exponentially weighted moving average smooths a noisy signal by giving more weight to recent observations:

$$S_t = \alpha x_t + (1 - \alpha) S_{t-1}, \quad \alpha \in (0, 1) \quad (1)$$

A higher α reacts faster to changes but amplifies noise; a lower α is more stable but slower to converge. Jacobson [9] originally applied EWMA to TCP round-trip-time estimation. The same technique has been adopted for load balancing by Envoy [7], where a separate EWMA per upstream tracks recent request latency.

C. Health Checking and Failure Detection

Active health checking — periodically pinging each backend with a lightweight request — is the standard mechanism for failure detection in reverse proxies such as NGINX [1] and HAProxy [2]. The trade-off between check frequency and the window of forwarded failures is well understood [4]. Less commonly studied is the interaction between health-check latency and routing-signal estimation: if a probe is issued to an idle worker, the measured latency reflects idle network RTT, not true request-processing capacity.

D. Containerised Testbeds

Docker Compose has become the standard tool for reproducible multi-service experiments on a single host [10]. Kubernetes [3] extends this to multi-host clusters but adds scheduling complexity that obscures the routing algorithm’s effect. We use Docker Compose to isolate the routing algorithm as the sole independent variable across runs.

III. PROBLEM FORMULATION

A. System Model

Let $W = \{w_1, w_2, \dots, w_n\}$ be a set of n backend workers. Each worker w_i has a true service-time distribution F_i for a given request type. In our testbed, $n = 3$; workers w_1 and w_2 are homogeneous with mean service time $\mu_{\text{fast}} \approx 45$ ms, and w_3 is degraded with $\mu_{\text{slow}} \approx 180$ ms. Worker capacities are unknown to the load balancer a priori and must be inferred from observations.

Requests arrive at the load balancer at rate λ (VUs generating requests with inter-arrival determined by response time and a fixed sleep). The load balancer maintains a state vector $\mathbf{s} = (\text{in_flight}_i, \hat{\mu}_i, h_i)_{i=1}^n$, where in_flight_i is the current number of outstanding requests, $\hat{\mu}_i$ is the estimated mean service time (via EWMA), and $h_i \in \{0, 1\}$ is the health status.

B. Routing Policies

A routing policy $\pi : \mathbf{s} \rightarrow W$ maps the current state to a worker selection. We evaluate three policies:

- 1) **Round-Robin:** $\pi_{\text{RR}}(\mathbf{s}) = w_{(k \bmod n)+1}$, where k increments atomically with each request and only healthy workers are counted.
- 2) **Least-Connections:** $\pi_{\text{LC}}(\mathbf{s}) = \arg \min_{w_i: h_i=1} \text{in_flight}_i$, with ties broken by $\hat{\mu}_i$.
- 3) **Latency-Aware (EWMA):** $\pi_{\text{LA}}(\mathbf{s}) = \arg \min_{w_i: h_i=1} (\hat{\mu}_i + \beta \cdot \text{in_flight}_i)$, where $\beta = 5$ ms is a penalty coefficient.

C. Optimisation Objectives

We evaluate each policy against three objectives, measured empirically:

- **Throughput** T (requests per second): higher is better.
- **Average and tail latency** ($\bar{L}$, L_{95}): lower is better.
- **Error rate** ε (fraction of requests timing out or receiving 5xx): lower is better.

These objectives are partly in tension: maximising throughput under overload requires accepting some increase in tail latency.

IV. PROPOSED SYSTEM DESIGN

A. Architecture Overview

The system follows a four-tier architecture, with all services running as Docker containers on a single host connected via a Docker bridge network.

- 1) **Workers** ($n = 3$): FastAPI async HTTP services that simulate analytics event processing with configurable latency.
- 2) **Load Balancer:** A custom FastAPI reverse proxy that implements the three routing policies, performs health checking, tracks EWMA latencies, and exposes Prometheus metrics.
- 3) **Observability:** Prometheus scrapes `/lb/metrics` every 2s; Grafana provides real-time dashboards.
- 4) **Load Generator:** k6 virtual users drive HTTP traffic according to configurable VU ramp profiles.

A supplementary *test-runner* service wraps k6 as a subprocess and exposes an HTTP API so that experiments can be triggered from the browser dashboard without SSH access.

B. Worker Service Design

Each worker exposes three endpoints:

- GET /api/v1/analytics/event?delay_ms=&cpu_ms=
— processes a synthetic analytics event. The handler first executes a synchronous busy-loop for `cpu_ms` milliseconds (blocking the `asyncio` event loop), then awaits `asyncio.sleep(delay_ms/1000)`.
- GET /api/v1/health — returns 200 OK immediately; used by the health-check loop.
- POST /api/v1/crash — calls `os._exit(1)`, simulating an abrupt process failure for the failure-injection scenario.

Worker heterogeneity is introduced via the `SLOWDOWN_FACTOR` environment variable, which multiplies both `delay_ms` and `cpu_ms` before processing. Worker-3 uses `SLOWDOWN_FACTOR=4`, yielding $4\times$ longer response times than workers 1 and 2.

C. Load Balancer Design

The load balancer is a FastAPI application with a single catch-all route that proxies all non-management requests to a selected worker. Per-worker state is maintained in a dataclass:

```
@dataclass
class Worker:
    name: str
    base_url: str
    healthy: bool = True
    in_flight: int = 0
    ewma_latency_ms: float = 100.0
    total_requests: int = 0
    failed_requests: int = 0
```

The `select_worker()` coroutine filters to healthy workers, then applies the active routing policy. A background `health_loop()` coroutine concurrently pings all workers every `HEALTH_CHECK_INTERVAL_MS` (default 1000 ms) with a timeout of `HEALTH_TIMEOUT_MS` (default 800 ms), updating the `healthy` flag on failure or recovery.

V. IMPLEMENTATION

A. Routing Algorithm Implementation

The three policies share a common `select_worker()` entry point that dispatches based on the `ALGORITHM` environment variable. Round-robin uses a module-level counter protected by `asyncio.Lock` to prevent race conditions. Least-connections and latency-aware use `min(candidates, key=...)` over the healthy worker list.

B. EWMA Update Correctness

A critical implementation detail is that the EWMA *must not* be updated during health checks. Health probes are issued to potentially idle workers and measure network RTT rather than request service time. If health-check latency is folded into the routing EWMA, all workers converge to similar low latency values during idle periods, eliminating the signal that allows latency-aware routing to prefer fast workers.

The correct update location is inside `proxy_to_worker()`, after the upstream response is received:

```
duration_ms = (time.perf_counter() - started) * 1000
worker.ewma_latency_ms = ewma(worker.ewma_latency_ms,
                               duration_ms)
```

The `run_health_check()` function updates only `worker.healthy` and `worker.last_health_check_at`.

C. In-Flight Counter Race Condition

In Python’s `asyncio` model, every `await` expression is a *yield point* at which the event loop may schedule another coroutine. The in-flight counter must be incremented *before* any such yield point in the request-handling path. In particular, `await request.body()` reads the request body and yields; multiple concurrent coroutines that each call `select_worker()` before any increments the counter will all see `in_flight=0` for all workers and may route to the same worker simultaneously.

The fix is to increment the counter immediately after worker selection:

```
worker = select_worker() # select before await
worker.in_flight += 1 # increment immediately
worker.total_requests += 1
stats.total_requests += 1
body = await request.body() # yield point (safe to increment)
```

D. Experiment Script Correctness

Each experiment scenario restarts the full Docker Compose stack with a different `ALGORITHM` environment variable. The routing policy is read by the load balancer at startup via `os.getenv()`. A subtle issue was that the `docker compose --profile loadtest run` command (which launches k6 as a one-off container) was initially invoked *without* the `ALGORITHM` variable prefix. Docker Compose resolved the missing variable to its default (`round_robin`) and recreated the load-balancer container, overwriting the intended algorithm. This was fixed by prefixing every `docker compose` invocation in the experiment loop with the appropriate variable:

```
ALGORITHM="$algorithm" docker compose \
--profile loadtest run --rm k6 run ...
```

E. Observability and Dashboard

The load balancer exposes a Prometheus text-format metrics endpoint at `/lb/metrics` with per-worker gauges (`in_flight`, `ewma_latency_ms`, `healthy`, `score`) and system-wide counters (`lb_requests_total`, `lb_failed_requests_total`, `rolling p95/p99`). A static HTML dashboard at `/dashboard` polls these endpoints and additionally allows operators to trigger k6 scenarios from the browser, streaming live output via a long-polling endpoint (`/tests/output?since=N`).

VI. EXPERIMENTAL METHODOLOGY

A. Environment

All experiments run on a personal home server equipped with an Intel Core i5-7500T CPU (2.70 GHz, 4 cores) and 15 GB RAM, running Debian Linux with Docker 27 and Docker Compose v2. Container-to-container network latency is sub-millisecond and negligible compared to the workload delays. Each experiment scenario is fully isolated: the entire stack is stopped (`docker compose down`), rebuilt, and restarted between runs to guarantee clean state and fresh worker EWMA values. Results are collected from k6 summary JSON exports and from the load balancer’s `/lb/stats` and `/lb/workers` endpoints, captured immediately after each k6 run completes.

B. Workload Parameters

All scenarios use `delay_ms=40` and `cpu_ms=5`. For workers 1 and 2 this yields approximately 45 ms per request (5 ms synchronous CPU work, 40 ms async sleep). For worker-3 (`SLOWDOWN_FACTOR=4`) this becomes approximately 180 ms (20 ms CPU, 160 ms sleep). The load balancer’s request timeout is 2000 ms; requests not completed within this window return a 502 error.

C. Scenarios

S1 — Constant Load. 30 virtual users, 60 s duration. A steady-state baseline that allows EWMA values to converge and measures the equilibrium traffic distribution and latency for each algorithm.

S2 — Traffic Spike. VUs ramp from 5 to 80 over 5 s, hold for 30 s, then ramp back to 0. This tests algorithm responsiveness when concurrency increases abruptly. The ramp is fast relative to EWMA convergence, creating a transient period of less-informed routing.

S3 — Stress Test. VUs ramp from 5 to 300 over 70 s. This progressively saturates the system and exposes the point at which each algorithm transitions from stable to degraded operation. At 300 VUs the system is intentionally overloaded.

S4 — Failure Injection. Constant 30 VUs using the latency-aware algorithm for 60 s; worker-2 is terminated via its `/api/v1/crash` endpoint at $t = 20$ s. This measures detection latency and the fraction of requests lost during the failure transition.

D. Metrics and Collection

Primary metrics are collected by k6 and include `http_req_duration` (average and p95), `http_reqs` (throughput), and `http_req_failed` (error rate). Secondary metrics are collected from the load balancer’s JSON endpoints: `per-worker total_requests`, `ewma_latency_ms`, and `healthy status`. All raw results are stored as JSON files and parsed by a Python script (`scripts/parse_results.py`) to produce the tables in this paper.

VII. RESULTS

A. S1: Constant Load

Table I shows results for 30 VUs over 60 s.

TABLE I
CONSTANT LOAD RESULTS (30 VUS, 60 s)

Metric	RR	LC	LA
Avg latency (ms)	107.1	79.7	74.4
p95 latency (ms)	224.9	220.1	182.4
Throughput (req/s)	143.9	165.7	170.7
Error rate (%)	0.0	0.0	0.0
Traffic to worker-3 (%)	33.3	13.4	4.9
Jain’s fairness index J	1.000	0.847	0.733

All three algorithms achieve zero errors under moderate load. However, traffic distributions differ markedly. Round-robin sends 33.3% to each worker as designed. Least-connections reduces worker-3’s share to 13.4%: the slow worker’s 180 ms service time means it holds more in-flight connections per unit time, making it the least-preferred choice. Latency-aware is the most aggressive, sending only 4.9% to worker-3 — whose EWMA score (~200 ms) is four times that of the fast workers (~49 ms).

Performance follows distribution: latency-aware achieves 74.4 ms average latency, 30% below round-robin’s 107.1 ms, with 19% higher throughput (170.7 vs. 143.9 req/s). Least-connections is intermediate (79.7 ms, 165.7 req/s).

B. S2: Traffic Spike

Table II shows spike results. The burst to 80 VUs doubles or triples throughput for adaptive algorithms while keeping error rates at zero; round-robin lags significantly.

TABLE II
SPIKE TEST RESULTS (5 → 80 VUS)

Metric	RR	LC	LA
Avg latency (ms)	392.1	182.0	172.2
p95 latency (ms)	1401.8	485.3	362.3
Throughput (req/s)	114.5	216.3	225.8
Error rate (%)	0.0	0.0	0.0
Traffic to worker-3 (%)	33.3	15.4	12.0
Jain’s fairness index J	1.000	0.871	0.830

Round-robin’s p95 of 1402 ms indicates queuing at worker-3: fast workers cannot process requests fast enough to avoid

backlog when one-third of traffic goes to a 4 $\times$ -slower worker. Adaptive algorithms limit this to 12–15%, keeping p95 under 500 ms and nearly doubling throughput (216–226 vs. 114 req/s).

C. S3: Stress Test

Table III shows stress results at saturation.

TABLE III
STRESS TEST RESULTS (5 $\rightarrow$ 300 VUs)

Metric	RR	LC	LA
Avg latency (ms)	595.4	440.9	440.8
p95 latency (ms)	2524.7	2601.7	2276.5
Throughput (req/s)	173.3	231.7	228.7
Error rate (%)	7.59	1.63	0.80
Traffic to worker-3 (%)	33.3	16.8	12.7
Jain’s fairness index J	1.000	0.891	0.839
Workers healthy at end	None	All	All

Round-robin saturates all three workers evenly, cascading into health check failures (all workers marked unhealthy by test end) and 7.6% errors. Both adaptive algorithms keep all workers healthy and achieve 32% higher throughput with 5–10 $\times$ lower error rates. Latency-aware has a slight edge over least-connections in error rate (0.80% vs. 1.63%) and p95 (2277 ms vs. 2602 ms).

D. S4: Failure Injection

Table IV shows failure-injection results using the latency-aware algorithm.

TABLE IV
FAILURE INJECTION RESULTS (LA, 30 VUs, WORKER-2 CRASH AT $t=20$ s)

Metric	Value
Detection latency	< 1 s (1 health-check interval)
k6 avg latency (full 60 s)	94.1 ms
k6 p95 latency	241.5 ms
Throughput	153.6 req/s
Total errors (LB stats)	1 of 9252 requests (0.01%)
Worker-1 (post-crash)	6 462 requests, 0 failures
Worker-2 (pre-crash only)	1 585 requests, 1 failure
Worker-3 (post-crash)	1 205 requests, 0 failures

Worker-2’s crash is detected within one health-check interval. Only one request fails (a 502 proxy error during the in-progress connection at crash time). The latency-aware algorithm subsequently routes 84% of new traffic to worker-1 (EWMA $\approx$ 67 ms) and 16% to worker-3 (EWMA $\approx$ 215 ms), maintaining 94 ms average latency throughout.

E. Resource Utilization (30 VUs, Constant Load)

Table V shows container-level CPU and memory usage, sampled via `docker stats` at mid-test (after EWMA convergence) under the constant-load workload. CPU percentages are per-core (100 % = one fully used CPU core).

The most striking result is worker-3’s CPU usage: under round-robin it reaches 98.5% (one saturated core), while under

TABLE V
RESOURCE UTILIZATION PER CONTAINER (30 VUs, CONSTANT LOAD)

Container	CPU (%)		Mem (MiB)	
	Idle	Under load	Idle	Under load
<i>Round-Robin</i>				
Load balancer	1.2	29.3	46.5	46.4
Worker-1	0.3	26.3	32.9	32.9
Worker-2	0.3	26.6	40.5	40.5
Worker-3	0.2	98.5	32.8	32.9
<i>Least-Connections</i>				
Load balancer	1.2	48.1	46.5	42.4
Worker-1	0.3	45.4	32.9	32.9
Worker-2	0.3	39.3	40.5	40.5
Worker-3	0.2	42.0	32.8	32.9
<i>Latency-Aware</i>				
Load balancer	1.2	45.7	46.5	44.4
Worker-1	0.3	46.9	32.9	32.9
Worker-2	0.3	47.9	40.5	40.5
Worker-3	0.2	18.6	32.8	32.9

latency-aware it drops to 18.6%. This directly reflects the traffic share (33% vs. 4.9%) and confirms that the algorithm prevents the CPU hotspot that round-robin creates. Memory footprint is stable across all algorithms, with no growth during the test, confirming the absence of memory leaks.

The load balancer itself consumes more CPU under adaptive algorithms (48% for LC/LA vs. 29% for RR). However, adaptive algorithms also handle more requests (166–171 vs. 144 req/s), so the per-request LB overhead is approximately 0.29 %/req (LC), 0.27 %/req (LA), and 0.20 %/req (RR) — a 35–45% overhead increase that is negligible in absolute terms with $n = 3$ workers.

VIII. INTEGRATED SYSTEM REQUIREMENTS AND CONFORMANCE SPECIFICATION

A. System Requirements

Table VI lists the system requirements using formal wording: **shall** for mandatory, **should** for recommended, and **may** for optional capabilities.

B. Conformance Checks

Table VII lists the five conformance checks that directly link requirements to experimental evidence. Each check specifies a pass criterion derived from the measured results.

IX. DISCUSSION

A. Key Findings

Adaptive algorithms consistently outperform round-robin. Across all four scenarios, both LC and LA reduce average latency, increase throughput, and — under stress — cut failure rates by 5–10 $\times$. The benefit is present even at moderate load (30 VUs), not only at saturation. This is because any traffic sent to the slow worker imposes a latency penalty on the client that issued the request; minimising that share directly reduces average latency.

TABLE VI
SYSTEM REQUIREMENTS AND CONFORMANCE SPECIFICATION

ID	Requirement	Pri.	Verification	Evidence
R1	The system shall maintain at least three concurrent backend workers.	Must	Docker Compose ps	All four scenarios run with workers 1–3 active.
R2	The load balancer shall implement round-robin routing as a selectable algorithm.	Must	Functional: uniform distribution check	S1 shows 33.3%/33.3%/33.3% under RR.
R3	The load balancer shall implement least-connections routing.	Must	Functional: worker-3 receives fewer requests than under RR	S1: LC routes 13.4% to worker-3 vs. 33.3% for RR.
R4	The load balancer shall implement latency-aware (EWMA) routing.	Must	Functional: worker-3 share lower than LC	S1: LA routes 4.9% to worker-3, lowest of all algorithms.
R5	The system shall perform active health checks on all workers at most every 1 s.	Must	Inspect health-check log timestamps	Health loop configured at 1000 ms interval; verified in code.
R6	Unhealthy workers shall be excluded from routing within 1 s of failure detection.	Must	Failure injection (S4)	S4: worker-2 excluded after 1 interval; only 1 request failed.
R7	The load balancer shall expose Prometheus-compatible metrics at /lb/metrics.	Must	curl /lb/metrics format check	Metrics file collected after each scenario; Grafana dashboard live.
R8	The system shall support synthetic workload injection via k6.	Must	k6 summary JSON present after each run	37 result files produced across all scenarios.
R9	Workers shall support a configurable SLOWDOWN_FACTOR to model heterogeneity.	Must	Compare EWMA: worker-3 $\approx 4\times$ workers 1–2	All scenarios: worker-3 EWMA ≈ 190 – 200 ms vs. ≈ 49 ms.
R10	The system should support deliberate worker crash for failure testing.	Should	POST /api/v1/crash returns; container stops	S4: worker-2 crashed at $t = 20$ s; marked unhealthy within 1 s.
R11	The load balancer should track per-worker EWMA latency for routing decisions.	Should	Inspect ewma_latency_ms field in /lb/workers	Tracked and verified across all scenarios; drives LA routing.
R12	The system may expose a browser-accessible dashboard for live monitoring and test control.	Optional	Manual browser test at /dashboard	Dashboard deployed; accessible at pcd.razc.ro.

Latency-aware routing most aggressively avoids slow workers. By combining EWMA latency with in-flight count in a single score, LA sends only 5–13% of traffic to worker-3 across all scenarios, compared to 13–17% for LC and 33% for RR. The EWMA score encodes both current load (via in-flight) and historical speed (via EWMA), making it more selective than in-flight count alone.

Stress testing exposes correctness requirements. At 300 VUs, round-robin’s equal distribution overloads worker-3, whose slow responses cause request timeouts, which cause health-check failures, which cascade into all three workers being marked unhealthy. Adaptive algorithms, by directing less traffic to worker-3, avoid this cascade.

Failure recovery is near-instantaneous. With a 1 s health check interval, the mean time to isolate a crashed worker is at most 1 s. In our experiment, only one request was lost. After isolation, the latency-aware algorithm correctly prefers

the faster surviving worker (worker-1 at ~ 67 ms over worker-3 at ~ 215 ms).

Fairness and performance trade off against each other. Jain’s fairness index $J = (\sum x_i)^2 / (n \sum x_i^2)$ measures how evenly traffic is distributed across workers, with $J = 1$ being perfectly equal. Round-robin achieves $J = 1.000$ by design, but this fairness comes at the cost of routing to the slow worker at its full equal share. Least-connections reduces fairness to $J = 0.847$ – 0.891 across scenarios; latency-aware to $J = 0.733$ – 0.839 . The lower fairness values directly correspond to better performance: fewer requests reach the slow worker, reducing average latency and increasing throughput. In heterogeneous deployments, equal traffic distribution is not a goal — proportional utilisation of actual capacity is.

Implementation correctness is as important as algorithm choice. The two bugs described in Section V each individually caused all algorithms to produce identical 33% distributions.

TABLE VII
CONFORMANCE CHECKS

ID	Check	Req.	Pass Criterion
C1	Algorithm differentiation under constant load	R2, R3, R4	Worker-3 traffic: RR $\approx 33\%$, LC $< 20\%$, LA $< 10\%$ (S1 confirmed: 33.3%, 13.4%, 4.9%).
C2	Failure detection within one interval	R5, R6	Fewer than 2 failed requests after a worker crash at $t = 20$ s (S4: 1 failure).
C3	Adaptive algorithms outperform RR under stress	R3, R4	Error rate $< 2\%$ and throughput > 200 req/s for LC and LA at 300 VUs (S3: LC 1.63%/231 rps, LA 0.80%/229 rps; RR 7.59%/173 rps).
C4	Prometheus metrics endpoint conforms to text format	R7	<code>curl /lb/metrics</code> produces valid Prometheus exposition with per-worker labels (verified; Grafana ingests successfully).
C5	Experiment reproducibility	R8, R9	Running <code>bash scripts/run-experiments.sh</code> produces 37 result files covering all algorithm-scenario combinations without manual intervention.

This result is counter-intuitive: both are concurrency correctness issues that are invisible in low-load unit tests but dominant at moderate concurrency. They serve as a cautionary example that correctness and performance are not separable concerns in async systems.

B. Algorithm Trade-offs

Throughput vs. tail latency. All algorithms exhibit high p95 values at 300 VUs (> 2 s) because the system is overloaded. Adaptive algorithms improve average latency but cannot eliminate tail latency once fast workers are also saturated.

Computational overhead. Round-robin is $O(1)$ per request; LC and LA are $O(n)$ over healthy workers. With $n = 3$ the overhead is negligible. At hundreds of backends, a min-heap would be required.

EWMA convergence warmup. With $\alpha = 0.25$, roughly 10–15 requests are needed to converge from the 100 ms initial value. During this warmup period LA behaves closer to round-robin. Frequent restarts or very short-lived services amplify this effect.

X. SECURITY, PRIVACY, SAFETY, AND ETHICAL CONSIDERATIONS

A. Security

The load balancer exposes several sensitive endpoints that must be access-controlled in any deployment beyond a private network:

- `POST /api/v1/crash` terminates a worker process. In production this endpoint must be removed or restricted to authenticated operators only, as unauthenticated access constitutes a trivial denial-of-service vector.
- `GET /lb/stats` and `GET /lb/workers` reveal internal topology (worker base URLs, error rates, EWMA latency values). This information aids an attacker in identifying degraded nodes or predicting routing behaviour for targeted traffic shaping.
- The test-runner service executes k6 as a subprocess. The scenario parameters (`vus`, `duration`) are passed to k6 via environment variables; input validation must ensure these cannot be used for command injection or resource exhaustion.

The current deployment at `pcd.razc.ro` is accessible over HTTPS (via a wildcard Let's Encrypt certificate) but has no authentication layer. This is acceptable for a course prototype but would require at minimum HTTP Basic Auth or OAuth before any production use.

B. Privacy

The system processes synthetic analytics events with no real user data. Prometheus stores metric time-series (request counts, latencies, worker health) which are aggregated and contain no personally identifiable information. Log files record HTTP request paths and source IP addresses; in a production system these would require data retention policies in compliance with applicable regulations (e.g., GDPR).

C. Safety

The `/api/v1/crash` endpoint is intentionally destructive. In a real infrastructure deployment, calling this endpoint on a production worker would cause service disruption. The endpoint is useful for controlled chaos engineering but must be gated behind a change-management process (e.g., maintenance window, approval workflow) in production environments.

D. Ethical Considerations

All load tests were conducted on infrastructure owned and operated by the authors. No public services or third-party systems were targeted. The k6 load generator and all dependent services are open-source. The experimental results and implementation are made publicly available to support reproducibility and future research.

XI. LIMITATIONS AND THREATS TO VALIDITY

A. Internal Validity

Single-host confounding. All containers share the same 4-core CPU and memory. At high VU counts, CPU scheduling contention between the k6 process, the load balancer, and three workers may alter results in ways that would not occur on a multi-host deployment. The relative ordering of algorithms is unlikely to change, but absolute latency and throughput values are hardware-specific.

Short experiment duration. Each run lasts 60–70 s. EWMA convergence, long-term steady-state distributions, and recovery from repeated transient failures cannot be studied at this timescale.

Single repetition. Each scenario was run once per algorithm. Without repeated runs we cannot report confidence intervals or test for statistical significance. Performance variability between runs on shared hardware may be non-negligible.

Synthetic workload. The `asyncio.sleep` + busy-loop model produces deterministic service times without the queue-length feedback or memory effects present in real analytics backends (e.g., database query plans, JVM JIT warmup, OS page cache effects).

B. External Validity

Generalisation to larger worker pools. With $n = 3$ workers, the routing decision space is small. With $n = 100$, the relative advantage of EWMA scoring over in-flight counting may change because there are more equally-fast alternatives to route to.

Generalisation to heterogeneous workloads. All requests carry the same `delay_ms` and `cpu_ms` parameters. In production analytics systems, request cost is often highly skewed (e.g., cold-start vs. cached queries), which would favour adaptive algorithms even more strongly.

Fixed EWMA α . The decay factor $\alpha = 0.25$ was chosen empirically. A systematic sweep across α values would be needed to characterise the convergence/stability trade-off and determine whether a different value would change the ranking of algorithms.

C. Construct Validity

EWMA at end of test as proxy for routing behaviour. The worker-level EWMA values reported in the results tables are captured at the end of each experiment and reflect the steady-state estimate, not the transient during the first 10–15 requests. During warmup, LA behaves closer to RR.

Rolling latency window. The p95/p99 values in `/lb/stats` cover only the last 1000 requests. For the stress test, where conditions change rapidly, this window reflects the final load level rather than the worst-seen during the ramp.

XII. CONCLUSION AND FUTURE WORK

We designed, implemented, and evaluated a containerised load-balancing testbed comparing round-robin, least-connections, and latency-aware EWMA-based routing across four controlled workload scenarios. The principal findings are:

- 1) **Adaptive algorithms outperform round-robin across all conditions.** Latency-aware routing reduces average latency by 30% and increases throughput by 19% at constant moderate load. Under extreme stress (300 VUs), both adaptive algorithms sustain 32% higher throughput with 5–10 $\times$ lower error rates.
- 2) **Failure recovery is near-instantaneous** with a 1 s health-check interval: the system loses only one request out of 9252 after a complete worker crash.
- 3) **Implementation correctness is critical.** Two concurrency bugs — EWMA contamination by health-check latency and in-flight counter increment after an `asyncio` yield point — caused all algorithms to produce identical routing distributions until fixed.

Future work directions include:

- **Shorter health-check intervals and circuit breakers.** A 200–500 ms health-check window combined with a circuit-breaker pattern (trip after k consecutive failures) would reduce the failure-exposure window further.
- **Thread-pool isolation for CPU-bound work.** Moving the synchronous busy-loop to `asyncio.run_in_executor` would decouple request processing from health-check responsiveness, making worker-3 a more realistic model of a slow but healthy backend.
- **Reinforcement-learning routing.** An RL agent that treats each request as an action could learn non-stationary workload patterns without explicit EWMA parameter tuning.
- **Systematic α sweep.** Evaluating EWMA decay factors across $\alpha \in \{0.1, 0.25, 0.5, 0.75\}$ would quantify the convergence–stability trade-off.
- **Statistical replication.** Running each scenario five or more times and reporting means with 95% confidence intervals would strengthen the validity of the conclusions.

REFERENCES

- [1] NGINX Inc., “NGINX Load Balancing,” <https://docs.nginx.com/nginx/admin-guide/load-balancer/http-load-balancer/>, accessed 2024.
- [2] HAProxy Technologies, “HAProxy Configuration Manual,” <https://docs.haproxy.org/>, accessed 2024.
- [3] The Kubernetes Authors, “Service – Kubernetes Documentation,” <https://kubernetes.io/docs/concepts/services-networking/service/>, accessed 2024.
- [4] A. S. Tanenbaum and M. Van Steen, *Distributed Systems: Principles and Paradigms*, 3rd ed. Pearson, 2017.
- [5] V. Cardellini, M. Colajanni, and P. S. Yu, “Dynamic load balancing on Web-server systems,” *IEEE Internet Computing*, vol. 3, no. 3, pp. 28–39, 1999.
- [6] V. S. Pai, M. Aron, G. Banga, M. Svendsen, P. Druschel, W. Zwaenepoel, and E. Nahum, “Locality-aware request distribution in cluster-based network servers,” in *Proc. ASPLOS*, 1998, pp. 205–216.
- [7] Envoy Proxy, “Load Balancing Policies,” https://www.envoyproxy.io/docs/envoy/latest/intro/arch_overview/upstream/load_balancing/load_balancers, accessed 2024.
- [8] M. Mitzenmacher, “The power of two choices in randomized load balancing,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 12, no. 10, pp. 1094–1104, 2001.
- [9] V. Jacobson, “Congestion avoidance and control,” *ACM SIGCOMM Comput. Commun. Rev.*, vol. 18, no. 4, pp. 314–329, 1988.
- [10] Docker Inc., “Docker Compose Overview,” <https://docs.docker.com/compose/>, accessed 2024.